

MeiFactor – Documento Técnico (n8n-first + Supabase)

Versão: 1.0 • Gerado em: 2025-10-29 00:00 UTC

Backend **n8n-first**: um único Main Gateway (Webhook) roteia para sub-workflows (Auth, Account, Forms, Report, Chat).
Banco **Supabase/Postgres** com **RLS** por `org_id` e **pgvector** p/ IA/RAG. Storage guarda uploads e PDFs.
Este doc define arquitetura, contratos de API, DDL, RLS, segurança, observabilidade, implantação e testes.

1. Objetivos e princípios

- **Simplicidade operacional**: tudo orquestrado no **n8n** (HTTP-in/HTTP-out).
- **Multi-tenant seguro**: isolamento por `org_id` com **Row Level Security**.
- **Escalabilidade progressiva**: sub-workflows isoláveis e versionáveis.
- **Custo baixo**: Supabase + IA local (Ollama/LM Studio); sem serviços pagos externos.
- **Observabilidade**: `requisitions` + `req_events` (auditoria e replay).
- **Padrão de respostas**: JSON { `ok`, `data|error` }; jobs longos retornam **202** + `req_id`.

2. Arquitetura (visão geral)

Componentes

- **Main Gateway (n8n)**: Webhook público; Code (normalização/autorização); Switch (`req.path` / `req.method`);
Execute Sub-workflow; Respond to Webhook; Error Trigger (global).
- **Sub-workflows por domínio**: `Auth`, `Account`, `Forms`, `Report`, `Chat`.
- **Supabase**: Postgres (com **pgvector**), Storage (buckets privados), RLS.
- **Agente de IA local**: Ollama/LM Studio (HTTP local), usado por Report/Chat.
- **Observabilidade**: `requisitions`, `req_events`, Error Workflow (alertas Slack/Email).

Fluxo do Gateway

1. Webhook (GET/POST) → 2) Code (normaliza/autoriza; grava `requisitions`) →
2. Switch (`/auth/*` , `/account/*` , `/forms/*` , `/report/*` , `/chat/*`) →
3. Execute Sub-workflow → 5) Respond (`200/202`) → 6) Error Trigger (alerta e auditoria).

Ambientes

- **DEV** (localhost) • **STG** (pré-prod) • **PRD** (prod).
 - Variáveis de ambiente exportadas no n8n:
`SUPABASE_URL` , `SUPABASE_SERVICE_ROLE_KEY` , `SUPABASE_ANON_KEY` ,
`JWT_SECRET` **ou** `JWT_PRIVATE_KEY` / `JWT_PUBLIC_KEY` (RS256),
`OLLAMA_URL` / `LMSTUDIO_URL` , `CLOUDFLARE_AI_TOKEN` (opcional).
-

3. Contratos de API (Gateway único)

Endpoint base: `https://api.meifactor.io/gateway/*`

Autorização: `Authorization: Bearer <token>` quando necessário.

Erros: `{ "ok": false, "error": { "code": "STRING", "message": "STRING", "details": {...} } }`.

3.1 Auth Service

- `POST /auth/request-code` → envia código (passwordless) para o e-mail do MEI.
Body: `{ "cnpj": "string" } → { "ok": true }.`
- `POST /auth/verify-code` → valida e cria sessão efêmera.
Body: `{ "cnpj": "string", "code": "123456" } → { "ok": true, "data": { "session_id": "..." } }.`
- `POST /auth/create-token` → emite **JWT 48h** com `{ sub, org_id, scopes, iat, exp }`.
Body: `{ "session_id": "..." } → { "ok": true, "data": { "access_token": "...", "expires_in": 172800 } }.`
- `POST /auth/verify-token` → valida assinatura/expiração/escopos + revogação.
- `POST /auth/update-token` → refresh/rotação (revoga token antigo).

3.2 Account Service

- `POST /account` → cria **organization** (CNPJ) e **user** (e-mail) e agenda job de **enriquecimento por CNPJ**.
- `GET /account/me` → dados de perfil/org do usuário autenticado.

- `PATCH /account` → atualização parcial.
- `DELETE /account` → remoção (soft-delete recomendado).

3.3 Forms Service (ingestão)

- `POST /forms/submit` → recebe JSON + arquivos (planilhas, PDFs, imagens, vídeos).
 - salva no **Storage** (`forms/`) e registra `uploads`.
 - cria **requisition** com `status='received'` para pipeline assíncrono.

3.4 Report Service

- `POST /report/run` → cria **requisition**, responde **202** + `req_id`.
- `GET /report/:id` → status; se `done`, retorna metadados + `pdf_url` em `reports/`.

3.5 Chat Service

- `POST /chat/send` → cria/atualiza `thread` + `message` (com anexos).
- `GET /chat/thread/:id` → histórico e status (opcional RAG com anexos indexados).

4. Modelo de dados (Supabase/Postgres)

Multi-tenant por `org_id`; **RLS** em todas as tabelas; **pgvector** para embeddings.

Buckets: `forms/`, `reports/`, `chat/` (privados; URLs assinadas).

4.1 DDL (principal)

```
-- Extensões
create extension if not exists "uuid-ossf";
create extension if not exists pgcrypto;
create extension if not exists vector;

-- Organizações (MEI)
create table organizations (
  id uuid primary key default uuid_generate_v4(),
  cnpj text unique not null,
  name text,
  plan text default 'free',
  created_at timestampz default now()
```

```

);

-- Usuários
create table users (
  id uuid primary key default uuid_generate_v4(),
  org_id uuid not null references organizations(id) on delete cascade,
  email text unique not null,
  role text default 'owner',
  password_hash text,
  created_at timestamptz default now()
);

-- Perfil / dados públicos do CNPJ
create table profiles (
  user_id uuid primary key references users(id) on delete cascade,
  org_id uuid not null references organizations(id) on delete cascade,
  display_name text,
  public_cnpj_json jsonb,
  updated_at timestamptz default now()
);

-- Workflows registrados (opcional)
create table workflows (
  id uuid primary key default uuid_generate_v4(),
  org_id uuid not null references organizations(id),
  name text,
  n8n_workflow_id text,
  version text,
  status text default 'active'
);

-- Requisições (Gateway)
create table requisitions (
  id uuid primary key default uuid_generate_v4(),
  org_id uuid not null references organizations(id),
  service text not null,          -- 'auth'|'account'|'forms'|'report'|'c
  route text not null,
  method text not null,
  payload_json jsonb,
  status text not null default 'received',  -- received|processing|done|
  n8n_execution_id text,
  created_at timestamptz default now()
);

```

```

-- Eventos de execução
create table req_events (
  id uuid primary key default uuid_generate_v4(),
  req_id uuid not null references requisitions(id) on delete cascade,
  step text,
  node_key text,
  input_json jsonb,
  output_json jsonb,
  status text,
  error_msg text,
  created_at timestamptz default now()
);

-- Tokens
create table tokens (
  id uuid primary key default uuid_generate_v4(),
  org_id uuid not null references organizations(id),
  user_id uuid references users(id) on delete cascade,
  type text not null,          -- 'access'|'refresh'|'service'
  scopes text[] not null,
  hash text not null,         -- hash do token; não salve o token em
  expires_at timestamptz not null,
  created_at timestamptz default now(),
  revoked boolean default false
);

-- Accounts
create table accounts (
  id uuid primary key default uuid_generate_v4(),
  org_id uuid not null references organizations(id),
  email text not null,
  status text default 'active',
  metadata jsonb,
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);

-- Reports
create table reports (
  id uuid primary key default uuid_generate_v4(),
  org_id uuid not null references organizations(id),
  req_id uuid references requisitions(id),
  kind text,
  params_json jsonb,

```

```

    result_json jsonb,
    pdf_url text,
    status text default 'processing',
    generated_at timestamptz
);

-- Uploads
create table uploads (
    id uuid primary key default uuid_generate_v4(),
    org_id uuid not null references organizations(id),
    user_id uuid references users(id),
    bucket text not null,
    object_path text not null,
    media_type text,
    size_bytes bigint,
    sha256 text,
    created_at timestamptz default now()
);

-- Vetores (embeddings) para RAG
create table doc_chunks (
    id uuid primary key default uuid_generate_v4(),
    org_id uuid not null references organizations(id),
    upload_id uuid references uploads(id) on delete cascade,
    chunk_index int,
    content text,
    embedding vector(384), -- all-MiniLM-L6-v2 (384d)
    created_at timestamptz default now()
);

-- Índices
create index on doc_chunks using ivfflat (embedding vector_cosine_ops) wi
create index on requisitions (org_id, created_at desc);
create index on tokens (expires_at);

```

4.2 RLS (Row-Level Security)

```

-- Habilitar RLS
alter table organizations enable row level security;
alter table users enable row level security;
alter table profiles enable row level security;
alter table requisitions enable row level security;

```

```

alter table req_events enable row level security;
alter table tokens enable row level security;
alter table accounts enable row level security;
alter table reports enable row level security;
alter table uploads enable row level security;
alter table doc_chunks enable row level security;

-- Exemplo de política por org_id
create policy org_read on requisitions for select
using ( org_id = (current_setting('request.jwt.claims', true)::jsonb->>'c

create policy org_insert on requisitions for insert
with check ( org_id = (current_setting('request.jwt.claims', true)::jsonk

```

Observações

- Inclua `org_id` nos claims do JWT.
- Tokens de serviço **não** devem ser aceitos no front.
- Valide assinatura/expiração/escopos e **revogação** (`revoked=false`).

4.3 Buckets e políticas de Storage

- `forms/` , `reports/` , `chat/` → **privados**; use **URLs assinadas** para download.

5. Auth (JWT 48h + refresh; escopos)

Create: { `sub`, `org_id`, `scopes`, `iat`, `exp=48h` } + `sha256(token)` em `tokens.hash`,
`expires_at`, `revoked=false`.

Verify: assinatura/expiração/escopos + `revoked=false`.

Update: novo token e revoga anterior.

Escopos: `report:run`, `report:read`, `forms:write`, `chat:send`, `account:write`.

6. Orquestração no n8n (padrões e naming)

Naming: Workflows `GW_Main`, `SVC_Auth_CreateToken`, `SVC_Report_Run` ... Nodes `C_` (Code),
`HTTP_`, `DB_`, `SW_` (Switch), `RSP_` (Respond).

Boas práticas: idempotência (`Idempotency-Key`), rate limit simples, paginação, retries + Error Workflow.

7. Segurança

Secrets apenas no backend; validação de uploads (MIME/tamanho) e possível antivírus (ClamAV).
LGPD: consentimento, retenção, exclusão, auditoria.

8. Observabilidade e SRE

Logs com `n8n_execution_id`, `req_id`, `org_id`; Error Workflow global; replay de erros; métricas básicas.

9. Implantação

n8n + Ollama/LM Studio na mesma VM (Docker). Supabase gerenciado; migrações versionadas; backups semanais.

10. Testes

Coleções Postman/Insomnia; carga (k6/Artillery); seeds para DEV/STG.

11. Apêndice

Exemplo (assíncrono 202)

```
{ "ok": true, "req_id": "df159c7c-21ab-46b2-a4c1-9b2f7a3aa001" }
```

Consulta RAG (pgvector)

```
select content, 1 - (embedding <=> :query_embedding) as score
from doc_chunks
where org_id = :org_id
order by embedding <-> :query_embedding
limit 8;
```